CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 8

Name: Prionti Das
UID: 24BAI71005
Branch: CSE(AIML)
Section/Group: 24AIT_NTPP-3[B]
Subject: Full Stack- II
Date of Performance: 01/10/2026
Subject Code: 24CSP-337
Submitted to: Ms.Lovleen Kaur(E20454)

EXPERIMENT – 8.1

1. **AIM:** To implement reliability mechanisms in an event-driven system using retry strategies with exponential backoff, Dead Letter Queues (DLQ), and idempotency patterns to ensure reliable message processing and prevent duplicate event processing.

2. **OBJECTIVE:**
   1. To understand failure-handling mechanisms in distributed event-driven systems.
   2. To implement retry mechanisms using exponential backoff to handle failed message processing.
   3. To configure a Dead Letter Queue (DLQ) to store messages that fail after multiple retry attempts.
   4. To route events that fail after multiple retry attempts to the Dead Letter Queue (DLQ) for further analysis and debugging.
   5. To implement idempotency using event tracking to prevent duplicate message processing.

3. **SOFTWARE REQUIREMENTS:**
   1. Java Development Kit
   2. Spring Boot
   3. Docker
   4. Apache Kafka
   5. Apache Maven
   6. Visual Studio Code

4. **THEORY:**

   In distributed event-driven systems, failures may occur due to network problems, service downtime, invalid messages, or unexpected exceptions. Reliability mechanisms help handle these failures and ensure consistent message processing.Retry with exponential backoff attempts to process failed messages again, increasing the waiting time between retries. A Dead Letter Queue (DLQ) stores messages that continue to fail after the maximum retry attempts, allowing them to be analyzed and reprocessed later. Idempotency prevents duplicate processing by tracking unique event IDs and checking whether a message has already been processed.

   By combining these mechanisms with Apache Kafka and Spring Boot, the system becomes more reliable, fault-tolerant, and capable of handling failures without disrupting the processing of other messages.

5. **IMPLEMENTATION:**
   1. Install and configure Java JDK, Spring Boot, Apache Kafka, Docker, and Maven.
   2. Create a Spring Boot project and add the required Kafka dependencies.

24BAI71005_Prionti

3. Configure Kafka broker details and create the main event topic.
4. Implement a Kafka producer to publish events and a consumer to process them.
5. Configure a retry mechanism with exponential backoff to handle processing failures.
6. Create a Dead Letter Queue (DLQ) topic and route messages to it after all retry attempts fail.
7. Implement idempotency by tracking unique event IDs to prevent duplicate processing.
8. Run the application and test successful messages, failed messages, retry attempts, DLQ routing, and duplicate events.
9. Verify the results to ensure reliable and consistent message processing.

6. CODE :

```
package com.example.eventreliability.controller;
import com.example.eventreliability.model.EventMessage;
import com.example.eventreliability.model.FailureMode;
import com.example.eventreliability.service.EventReliabilityService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;
@RestController
@RequestMapping("/api/events")
public class EventController {
   private final EventReliabilityService service;
   public EventController(EventReliabilityService service) {
      this.service = service; }
   @PostMapping
   public ResponseEntity<Map<String, Object>> publish(
         @RequestBody EventMessage event) {
      if (event.getEventId() == null
            || event.getEventId().isBlank()) {
         return ResponseEntity.badRequest().body(
               Map.of("error", "eventId is required.")
         ); }
      if (event.getMessage() == null
            || event.getMessage().isBlank()) {

         return ResponseEntity.badRequest().body(
               Map.of("error", "message is required.")
         );}
      event.setEventId(event.getEventId().trim());
      event.setMessage(event.getMessage().trim());

      if (event.getFailureMode() == null) {
         event.setFailureMode(FailureMode.NONE);
      }
      Map<String, Object> result = service.publish(event);

      if ("DUPLICATE_SKIPPED".equals(result.get("status"))) {
         return ResponseEntity.ok(result);}
      return ResponseEntity.status(HttpStatus.ACCEPTED).body(result);  }
```

24BAI71005_Prionti

```
@GetMapping("/status")
public List<Map<String, Object>> statuses() {
   return service.getStatuses(); }
@GetMapping("/history")
public List<Map<String, Object>> history() {
   return service.getHistory(); }
@GetMapping("/dlq")
public List<Map<String, Object>> deadLetterEvents() {
   return service.getDeadLetterEvents();
}}
```

7. OUTPUT:

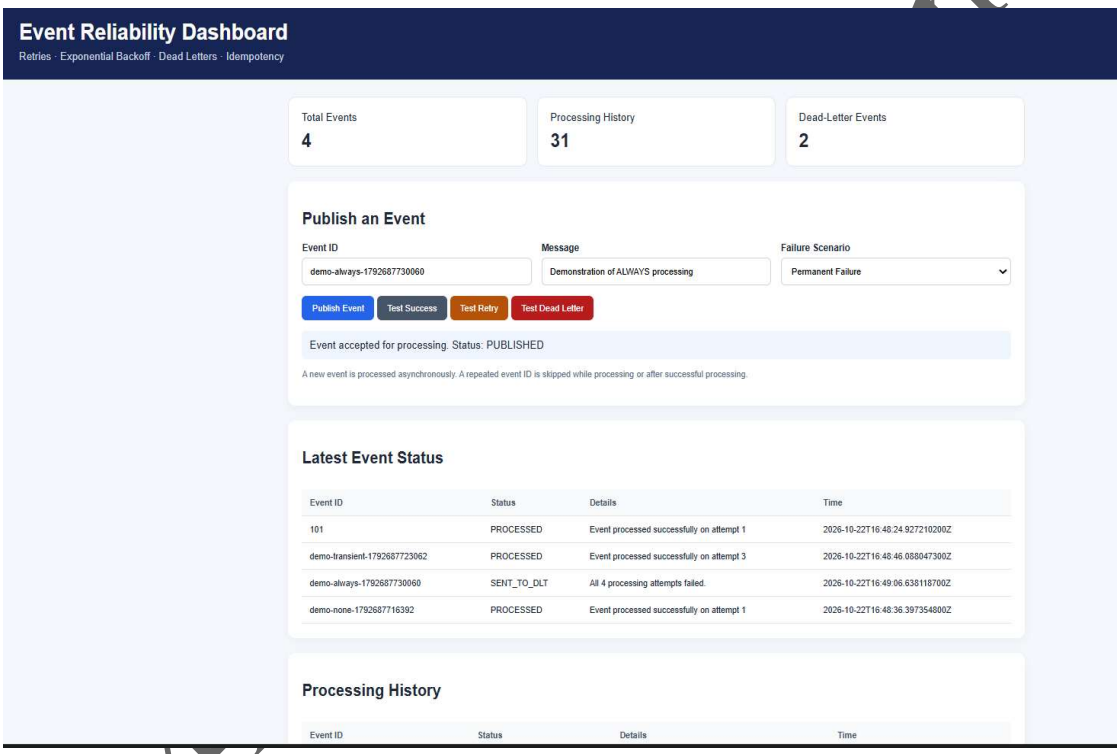


Fig 8.1:Event Reliability Dashboard Showing Retry Mechanisms, Dead-Letter Events and Idempotent Processing Status

The Event Reliability Dashboard displays the total number of events, processing history, and dead-letter events. It allows users to publish events and test different scenarios, including successful processing, temporary failures, and permanent failures. Failed events are retried automatically using retry mechanisms, while events that exceed the maximum retry attempts are sent to the Dead Letter Queue (DLQ). The dashboard also shows event IDs, processing statuses, details, and timestamps. It demonstrates reliable event processing and helps prevent duplicate processing through idempotency.

8. LEARNING OUTCOME:

- Understand failure handling in event-driven and distributed systems.
- Implement retry mechanisms using exponential backoff.
- Configure and use a Dead Letter Queue (DLQ) for failed messages.
- Apply idempotency to prevent duplicate message processing.
- Build more reliable and fault-tolerant Kafka-based applications.

24BAI71005_Prionti

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 8.2

1. **AIM:** To implement reliability mechanisms in an event-driven system using retry strategies with exponential backoff, Dead Letter Queues (DLQ), and idempotency patterns to ensure reliable message processing and prevent duplicate event processing.
2. **OBJECTIVE:**

    1. To understand failure-handling mechanisms in distributed systems.
    2. To implement retry mechanisms with exponential backoff to handle failed message processing.
    3. To configure a Dead Letter Queue (DLQ) to store messages that fail after multiple retry attempts.
    4. To route events that fail after multiple retry attempts to the Dead Letter Queue (DLQ) for further analysis and debugging.

3. **SOFTWARE REQUIREMENTS:**
    1. Java Development Kit
    2. Spring Boot
    3. Apache Kafka
    4. Docker
    5. Apache Maven
    6. Visual Studio Code
4. **THEORY:**

    In distributed event-driven systems, failures may occur due to network issues, service downtime, invalid messages, or unexpected exceptions. Reliability mechanisms help handle these failures and ensure consistent message processing.

    Retry with exponential backoff attempts to process failed messages again, increasing the waiting time between retries. A Dead Letter Queue (DLQ) stores messages that continue to fail after the maximum retry attempts so they can be analyzed and reprocessed later. Idempotency prevents duplicate processing by tracking unique event IDs and checking whether a message has already been processed.

    By combining these mechanisms with Apache Kafka and Spring Boot, the system becomes more reliable and fault-tolerant while maintaining consistent message processing.

5. **IMPLEMENTATION/PROCEDURE:**
    1. Install and configure Java JDK, Spring Boot, Apache Kafka, Docker and Maven.
    2. Create a Spring Boot project and add the required Kafka dependencies.
    3. Configure the Kafka broker and create the main event topic.
    4. Implement a Kafka producer to publish events and a consumer to process them.
    5. Implement a retry mechanism with exponential backoff to handle failed messages.
    6. Configure a Dead Letter Queue (DLQ) topic to store messages that fail after all retry attempts.
    7. Implement idempotency using unique event IDs to prevent duplicate message processing.
    8. Test successful messages, processing failures, retry attempts, DLQ routing, and duplicate events.
    9. Verify the results to ensure reliable and consistent message processing.

24BAI71005_Prionti

6. CODE:

```
package com.example.eventreliability.controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.Map;
@RestController
public class DemoInfoController {
   @GetMapping("/api/demo")
   public Map<String, String> getDemoInfo() {
      return Map.of(
            "studentName", "Prionti Das",
            "uid", "24BAI71005",
            "project", "Event Reliability System"
      ); }}
```

Prionti

7. OUTPUT:

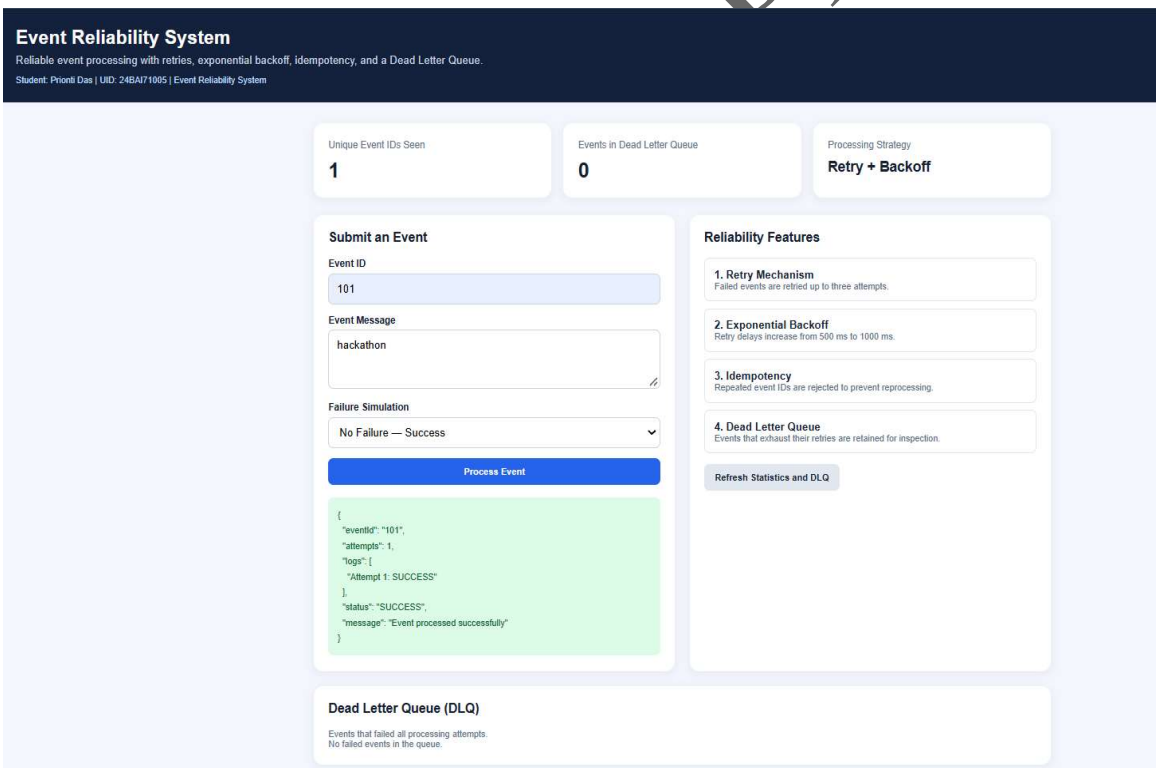


Fig 8.2: Event Reliability System Dashboard Demonstrating Retry Mechanism, Exponential Backoff, Idempotency, and Dead Letter Queue (DLQ)

The Event Reliability System dashboard allows users to submit events and simulate different failure scenarios. It displays the processing status, retry attempts, and event statistics. In the shown output, event ID `101` is processed successfully on the first attempt and the Dead Letter Queue (DLQ) contains no failed events. The system also demonstrates exponential backoff for retries and idempotency to prevent duplicate processing.

8. LEARNING OUTCOME:

- Understand failure-handling mechanisms in distributed event-driven systems.
- Implement retry strategies using exponential backoff.
- Configure a Dead Letter Queue (DLQ) to handle permanently failed messages.
- Apply idempotency to prevent duplicate message processing.
- Test failure scenarios and build reliable, fault-tolerant Kafka-based applications.

24BAI71005_Prionti